K8S Zabbix Reporter

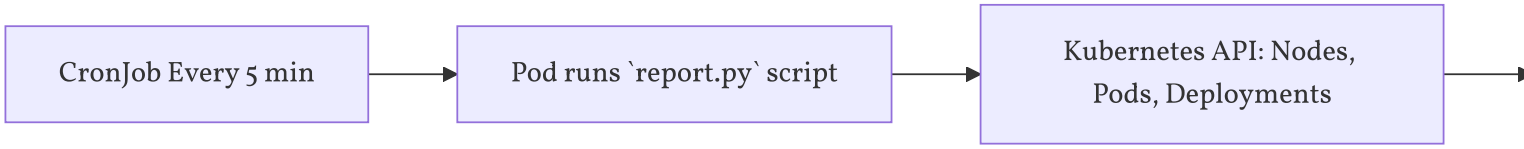
Summary

هدف: ارسال داده های کلاستر k8s به Zabbix برای دریافت Alert و ایجاد Dashboard.
خلاصه نحوه کارکرد: یک Resource از جنس CronJob یک Pod را هر ۵ دقیقه بالا می آورد. Pod با دسترسی های RBAC ای که به آن داده شده است با API کوبرنیتز یک لیست از Metric ها را استخراج کرده و به سرور Zabbix بر روی پورت ۱۰۰۵۱ می فرستد که با پروتکل Zabbix Trapper این کار انجام می شود. بعد از انجام این کار Pod با توجه به جنس Resource اش exit می شود.

محیط تستی: شبکه داخلی با IP های زیر:

- کلاستر: 192.168.0.12 تا 15
- Zabbix Server: 192.168.0.15

Data Flow:



لیست Metric های دریافتی:

Key	Meaning	Type
k8s.nodes.total	تعداد Node های کلاستر	unsigned
k8s.nodes.not_ready	تعداد Node های Unready	unsigned
k8s.pods.total	تعداد کل Pod های کلاستر	unsigned
k8s.pods.running / pending / failed / succeeded / unknown	وضعیت Pod ها	unsigned
k8s.pods.restarts	تعداد کل Restart های کل Pod ها	unsigned
k8s.pods.crashloop	تعداد Pod ها در وضعیت CrashLoopBackOff	unsigned
k8s.pods.crashloop.list	لیست Pod ها در وضعیت CrashLoopBackOff	text
k8s.deployments.total	تعداد Resource ها از جنس Deployment	unsigned
k8s.deployments.unavailable	تعداد Deployment هایی که Replica های آن کمتر از مقدار Desired است	unsigned

در Zabbix از Discovery استفاده شده است تا بتوانیم دیتا مربوط به هر تک Node را نیز بگیریم. برای وضعیتی که کلاستر به صورت Dynamic تعداد Node هایش از طریق Resource Pool ها زیاد و کم می شود مفید است:

Key	Meaning
k8s.node.ready[NODE]	مقدار ۱ برای Node در وضعیت Ready و ۰ برای خلاف آن
k8s.node.cpu_pct[NODE]	مقدار مصرف CPU بر اساس درصد
k8s.node.mem_pct[NODE]	مقدار مصرف Memory بر اساس درصد
k8s.node.disk_free_pct[NODE]	مقدار فضای باقی مانده در دیسک

Helm Chart

ساختار Helm Chart:

```
k8s-zabbix-reporter/
├── Chart.yaml
├── templates
│   ├── configmap.yaml
│   ├── cronjob.yaml
│   └── rbac.yaml
└── values.yaml
```

Root Level Helm Manifests

موارد استفاده شده برای `Chart.yaml` :

`Chart.yaml`

```yaml
apiVersion: v2
name: k8s-zabbix-reporter
description: CronJob that pushes node, pod and cluster data to zabbix trapper items
type: application
version: 0.1.0
appVersion: "1.0"
```

موارد استفاده شده برای `values.yaml` و توضیحات آن:

`values.yaml`

```yaml
schedule: "*/5 * * * *"

image:
  repository: python
  tag: 3.12-alpine
  pullPolicy: IfNotPresent

zabbix:
  server: "192.168.0.15"
  port: 10051
  host: k8s-cluster

resources:
  requests:
    cpu: 25m
    memory: 32Mi
  limits:
    cpu: 100m
    memory: 64Mi
```

Value	Effect
schedule	حاوی Cron نوشته شده برای اجرا Pod
image.*	حاوی Image و Tag مربوطه برای Container تعریف شده داخل Pod. از یک Image سبک پایتون استفاده شده است تا نیازی به استفاده از Custom Image نباشد. Image استفاده شده از نسخه Alpine استفاده می کند تا حجم آن کم باشد و در صورت پر بودن کلاستر یا وجود `ResourceQuota` بتوانیم Limit های منطقی و کمی را برای آن استفاده کنیم و تعداد Process های داخل Image زیاد نباشد تا نیاز به منابع بالا داشته باشد.
zabbix.server / port	آدرس IP و Port مربوط به Zabbix
zabbix.host	اسم Host ای که برای این نیازمندی باید در Zabbix ایجاد گردد

Value	Effect
resources	مقدار Resource Allocation تا Pod را به صورت `Burstable` تعریف کنیم

Templates

RBAC

تعریف اعمال دسترسی به Pod:

در `templates/rbac.yaml` تعداد ۳ Resource ایجاد می شود و بخاطر اینکه این دیتا ها باید از تمام Namespace ها دریافت شود به هیچ Namespace ای Bound نشده است. این `RoleBinding` فقط دسترسی های Read را به Pod می دهد.

`templates/rbac.yaml`

```
apiVersion: v1
kind: ServiceAccount
metadata:
   name: {{ .Release.Name }}
   namespace: {{ .Release.Namespace }}
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
   name: {{ .Release.Name }}
rules:
   - apiGroups: [""]
     resources: ["nodes","pods"]
     verbs: ["get","list"]
   - apiGroups: [""]
     resources: ["nodes/proxy"]
     verbs: ["get"]
   - apiGroups: ["apps"]
     resources: ["deployments"]
     verbs: ["list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
   name: {{ .Release.Name }}
roleRef:
   apiGroup: rbac.authorization.k8s.io
   kind: ClusterRole
   name: {{ .Release.Name }}
subjects:
   - kind: ServiceAccount
     name: {{ .Release.Name }}
     namespace: {{ .Release.Namespace }}
```

تعریف ۳ Resource تعریف شده به شکل زیر است:

- `ServiceAccount`:
  بگوییم با چه سطح دسترسی هایی می تواند در کلاستر کار کند و باید بعد از تعریف Pod برای این ایجاد شده است تا به Resource این `ClusterRole` نیز ایجاد کنیم. برای آن `ClusterRoleBinding`
- `ClusterRole`:
  دسترسی Read Only برای خواندن Metric های مربوطه مثل Endpoint ایجاد شده است. به Resource های کلاستر با این `api/v1/nodes/` دسترسی داده شده است.

- `ClusterRoleBinding`:
  ایجاد شده است. `ClusterRole` به `ServiceAccount` به منظور وصل کردن دسترسی های

ConfigMap

تعریف اضافه کردن کد به Pod:
این Resource کد مربوط به اپلیکیشن پایتونی که در Container اجرا می شود را در مسیر مربوطه قرار می دهد.

تعریف کد:
این کد ابتدا مقادیر Token و CA Certificate ای که به صورت اتوماتیک به Pod مربوطه به خاطر ServiceAccount اش Mount می شود را می خواند. بعد از آن با استفاده از Library های معمولی و Standard پایتون شروع به Request زدن به Endpoint های مربوط به API Server کرده و دیتا را دریافت می کند. تمام دیتا دریافتی به Zabbix ارسال می شود که اطلاعات مربوط به آن از `values.yaml` دریافت شده و در Container به صورت ENV قرار گرفته است. خروجی کد به صورت زیر است که می توانیم آن را با فرمان `kubectl logs -f` در Namespace مربوطه و با نام Pod ایجاد شده توسط Job ببینیم:

```
{"response":"success","info":"processed: 30; failed: 0; total: 30; seconds spent: 0.000604"}
```

`templates/configmap.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
    name: {{ .Release.Name }}-script
    namespace: {{ .Release.Namespace }}
data:
    report.py: |
        import os, ssl, json, socket, struct, urllib.request

        SA = "/var/run/secrets/kubernetes.io/serviceaccount/"
        API = "https://%s:%s" % (os.environ["KUBERNETES_SERVICE_HOST"],
os.environ["KUBERNETES_SERVICE_PORT"])
        TOKEN = open(SA + "token").read()
        CTX = ssl.create_default_context(cafile=SA + "ca.crt")
        ZHOST = os.environ["ZABBIX_HOST"]

        def get(path):
            req = urllib.request.Request(API + path, headers={"Authorization": "Bearer " +
TOKEN})
            return json.load(urllib.request.urlopen(req, context=CTX, timeout=20))

        def qty(v):
            v = str(v)
            units = {"Ki": 2**10, "Mi": 2**20, "Gi": 2**30, "Ti": 2**40, "m": 0.001}
            for u, m in units.items():
                if v.endswith(u):
                    return float(v[:-len(u)]) * m
            return float(v)

        data = []
        def put(key, value):
            data.append({"host": ZHOST, "key": key, "value": str(value)})

        # ---- Nodes ----
        nodes = get("/api/v1/nodes")["items"]
        put("k8s.node.discovery", json.dumps([{"{#NODE}": n["metadata"]["name"]} for n in
nodes]))
        not_ready = 0
        for n in nodes:
            name = n["metadata"]["name"]
```

```
            ready = any(c["type"] == "Ready" and c["status"] == "True" for c in n["status"]
["conditions"])
            not_ready += 0 if ready else 1
            put("k8s.node.ready[%s]" % name, 1 if ready else 0)
            try:
                s = get("/api/v1/nodes/%s/proxy/stats/summary" % name)["node"]
                cpu_cap = qty(n["status"]["capacity"]["cpu"])
                mem_cap = qty(n["status"]["capacity"]["memory"])
                put("k8s.node.cpu_pct[%s]" % name, round(s["cpu"]["usageNanoCores"] / 1e9 /
cpu_cap * 100, 2))
                put("k8s.node.mem_pct[%s]" % name, round(s["memory"]["workingSetBytes"] /
mem_cap * 100, 2))
                fs = s["fs"]
                put("k8s.node.disk_free_pct[%s]" % name, round(fs["availableBytes"] /
fs["capacityBytes"] * 100, 2))
            except Exception as e:
                print("stats failed for", name, e)
        put("k8s.nodes.total", len(nodes))
        put("k8s.nodes.not_ready", not_ready)

        # ---- Pods ----
        pods = get("/api/v1/pods")["items"]
        phases = {"Running": 0, "Pending": 0, "Failed": 0, "Succeeded": 0, "Unknown": 0}
        restarts, crash = 0, []
        for p in pods:
            phases[p["status"].get("phase", "Unknown")] = phases.get(p["status"].get("phase",
"Unknown"), 0) + 1
            for cs in p["status"].get("containerStatuses", []):
                restarts += cs.get("restartCount", 0)
                if cs.get("state", {}).get("waiting", {}).get("reason") == "CrashLoopBackOff":
                    crash.append("%s/%s" % (p["metadata"]["namespace"], p["metadata"]
["name"]))
        put("k8s.pods.total", len(pods))
        for k, v in phases.items():
            put("k8s.pods.%s" % k.lower(), v)
        put("k8s.pods.restarts", restarts)
        put("k8s.pods.crashloop", len(crash))
        put("k8s.pods.crashloop.list", ", ".join(crash) or "none")

        # ---- Deployments ----
        deps = get("/apis/apps/v1/deployments")["items"]
        bad = [d for d in deps if d["status"].get("availableReplicas", 0) <
d["spec"].get("replicas", 1)]
        put("k8s.deployments.total", len(deps))
        put("k8s.deployments.unavailable", len(bad))

        # ---- Send to Zabbix (trapper protocol) ----
        body = json.dumps({"request": "sender data", "data": data}).encode()
        pkt = b"ZBXD\x01" + struct.pack("<Q", len(body)) + body
        with socket.create_connection((os.environ["ZABBIX_SERVER"],
int(os.environ["ZABBIX_PORT"])), timeout=15) as sock:
            sock.sendall(pkt)
            buf = b""
            while len(buf) < 13:
                chunk = sock.recv(4096)
                if not chunk:
                    break
                buf += chunk
            rest = buf[13:]
            length = struct.unpack("<Q", buf[5:13])[0]
```

```
            while len(rest) < length:
                chunk = sock.recv(4096)
                if not chunk:
                    break
                rest += chunk
        print(rest.decode())
```

CronJob

تعریف:
یک CronJob به واسطه این Template بوجود می آید که در آن هر ۵ دقیقه یک Pod بوجود آمده که با دسترسی هایی که به API Server دارد Metric ها را دریافت کرده و به Zabbix Server ارسال می کند تا توسط Host آن برداشته شده و برای Monitoring استفاده شود.

`templates/cronjob.yaml`

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: {{ .Release.Name }}
  namespace: {{ .Release.Namespace }}
spec:
  schedule: {{ .Values.schedule | quote }}
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 1
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 0
      activeDeadlineSeconds: 50
      ttlSecondsAfterFinished: 300
      template:
        metadata:
          labels:
            app: {{ .Release.Name }}
        spec:
          serviceAccountName: {{ .Release.Name }}
          restartPolicy: Never
          securityContext:
            runAsNonRoot: true
            runAsUser: 65534
            seccompProfile:
              type: RuntimeDefault
          containers:
            - name: reporter
              image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
              imagePullPolicy: {{ .Values.image.pullPolicy }}
              command: ["python", "/scripts/report.py"]
              env:
                - name: ZABBIX_SERVER
                  value: {{ .Values.zabbix.server | quote }}
                - name: ZABBIX_PORT
                  value: {{ .Values.zabbix.port | quote }}
                - name: ZABBIX_HOST
                  value: {{ .Values.zabbix.host | quote }}
              resources:
                {{- toYaml .Values.resources | nindent 16 }}
              securityContext:
                allowPrivilegeEscalation: false
```

```
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: script
              mountPath: /scripts
      volumes:
        - name: script
          configMap:
            name: {{ .Release.Name }}-script
```

تعریف Directive های استفاده شده برای تعریف این CronJob:
خیلی از این موارد تعریف شده مثل `ttlSecondsAfterFinished` و `activeDeadlineSeconds` نیز می توانست به `values.yaml` اضافه شود که برای رساندن این تسک به Deadline ست شده انجام نشد:

Setting	Why
schedule	مقدار این Cron از `values.yaml` خوانده می شود
concurrencyPolicy: Forbid	اگر یک Run از این CronJob در حال اجرا باشد و تمام نشده باشد بعدی اجرا نمی شود
successfulJobsHistoryLimit: 1, failedJobsHistoryLimit: 3	دیتا آخرین Job موفق و دیتا ۳ Job ناموفق قبلی در صورت نیاز به رجوع به آن ها نگهداری می شود
backoffLimit: 0	در صورت fail شدن هیچ Retry ای انجام نمی شود.
activeDeadlineSeconds: 50	یک Run در صورت تمام نشدن بعد از ۵۰ ثانیه Terminate می شود
ttlSecondsAfterFinished: 300	یک Job و Pod آن بعد از ۳ دقیقه تمام شدن آن از کلاستر حذف می شود
serviceAccountName	نام `ServiceAccount` ای که سطح دسترسی Pod را مشخص می کند
Pod security context	دسترسی اجرا شدن این Pod با استفاده از root از آن گرفته شده است
Container security context	این Pod اجازه Escalate کردن دسترسی های خودش را ندارد و Filesystem آن نیز به صورت Read Only تعریف شده و تمام Capability های Linux آن نیز Drop شده است
Environment variables	تعریف ENV های مورد نیاز برای دسترسی به دیتا Zabbix
ConfigMap volume	تعریف مسیر Mount دیتای تعریف شده در ConfigMap

commands

در پایین تعدادی از فرمان هایی که برای چک کردن (linting) نصب و چک کردن Permission های این Pod اجرا شده است آورده شده. همچنین چندین Job به صورت Manual ایجاد شده است تا خروجی برای Zabbix ایجاد شود:

`linting`

```
helm lint ./k8s-zabbix-reporter
helm template t ./k8s-zabbix-reporter
helm template t ./k8s-zabbix-reporter --debug
helm template t ./k8s-zabbix-reporter > /tmp/out.yaml
kubectl apply --dry-run=server -f /tmp/out.yaml -n monitoring
```

`install`

```
helm install k8s-zabbix-reporter ./k8s-zabbix-reporter -n monitoring --create-namespace
helm list -n monitoring
kubectl -n monitoring get cronjob,networkpolicy,sa,cm
kubectl get clusterrole,clusterrolebinding | grep k8s-zabbix-reporter
```

`permissions check`

```
SA=system:serviceaccount:monitoring:k8s-zabbix-reporter
kubectl auth can-i list pods --all-namespaces --as=$SA
kubectl auth can-i list nodes --as=$SA
kubectl auth can-i get nodes/proxy --as=$SA
kubectl auth can-i list deployments.apps --all-namespaces --as=$SA
```

`manual job`

```
kubectl -n monitoring create job --from=cronjob/k8s-zabbix-reporter manual-test
kubectl -n monitoring get pods -w
kubectl -n monitoring logs job/manual-test
kubectl -n monitoring delete job manual-test
```

zabbix

یک Host با نام `k8s-cluster` تعریف و هیچ Interface ای برای آن تعریف نشد. Item های تعریف شده به صورت Plain Item در `hosts ->` `k8s-cluster -> items -> create items` تعریف شده اند.

یک Discovery Rule برای جنس Zabbix Trapper و برای این Host تعریف شده است و موارد آن به شکل زیر تعریف شده است:

Name	Key	Type of information	Units
Node {#NODE}: ready	k8s.node.ready[{#NODE}]	Numeric (unsigned)	
Node {#NODE}: CPU usage	k8s.node.cpu_pct[{#NODE}]	Numeric (float)	%
Node {#NODE}: memory usage	k8s.node.mem_pct[{#NODE}]	Numeric (float)	%
Node {#NODE}: disk free	k8s.node.disk_free_pct[{#NODE}]	Numeric (float)	%

با اجرای ابتدایی Manual Job کل Node ها به صورت Discovery تعریف شده اند.

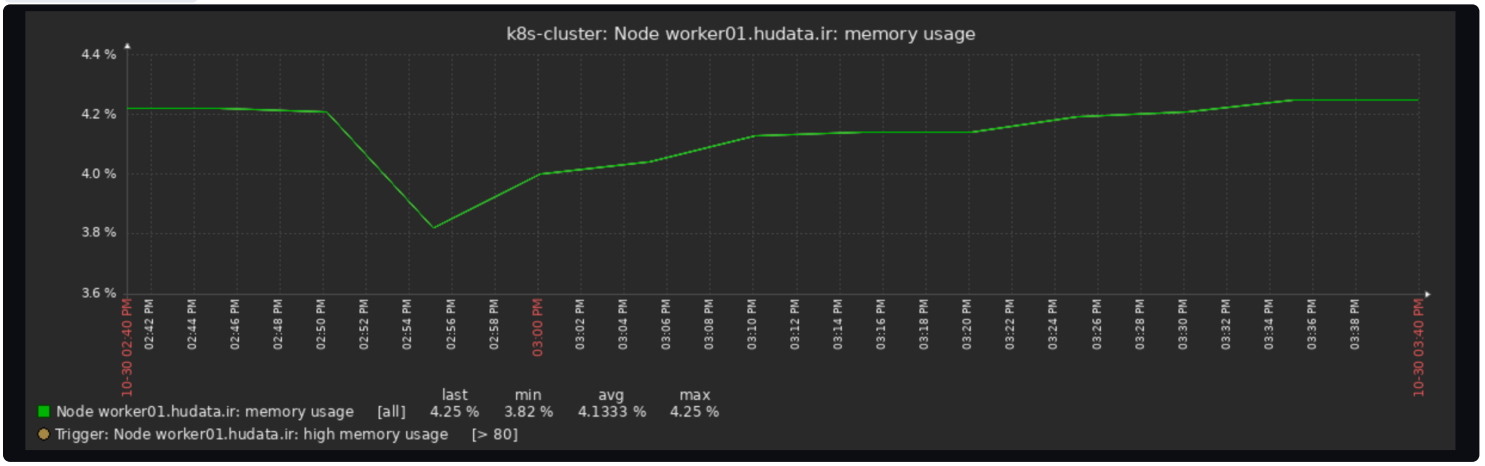
برای Zabbix یکسری Trigger های کلی و تعدادی Trigger Prototype به منظور ایجادPer node در Discovey تعریف شده است که لیست آن ها به شکل زیر است:

Name	Expression	Severity
Pod in CrashLoopBackOff	last(/k8s-cluster/k8s.pods.crashloop)>0	High
Pods stuck Pending	min(/k8s-cluster/k8s.pods.pending,15m)>0	Warning
Deployments unavailable	min(/k8s-cluster/k8s.deployments.unavailable,10m)>0	Warning
Nodes not ready	last(/k8s-cluster/k8s.nodes.not_ready)>0	Disaster
No data from reporter	nodata(/k8s-cluster/k8s.nodes.total,15m)=1	High

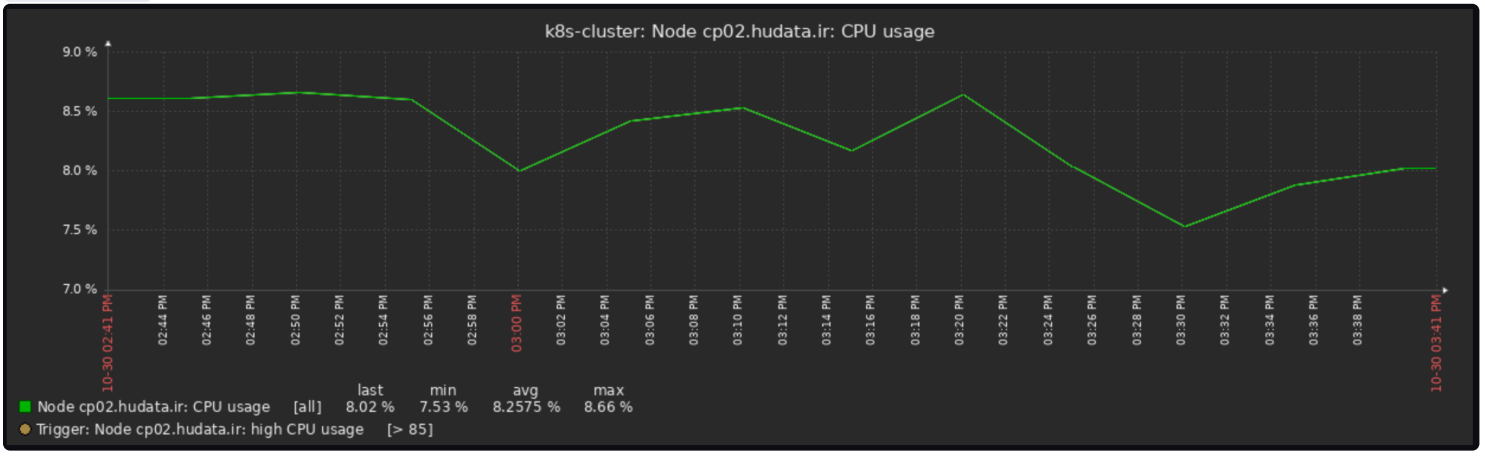
Name	Expression	Severity
Node {#NODE}: disk space below 10%	last(/k8s-cluster/k8s.node.disk_free_pct[{#NODE}])<10	High
Node {#NODE}: high memory usage	avg(/k8s-cluster/k8s.node.mem_pct[{#NODE}],10m)>80	Warning
Node {#NODE}: high CPU usage	avg(/k8s-cluster/k8s.node.cpu_pct[{#NODE}],10m)>85	Warning
Node {#NODE}: not ready	last(/k8s-cluster/k8s.node.ready[{#NODE}])=0	Disaster

تعدادی از Graph های ایجاد شده برای Monitoring:

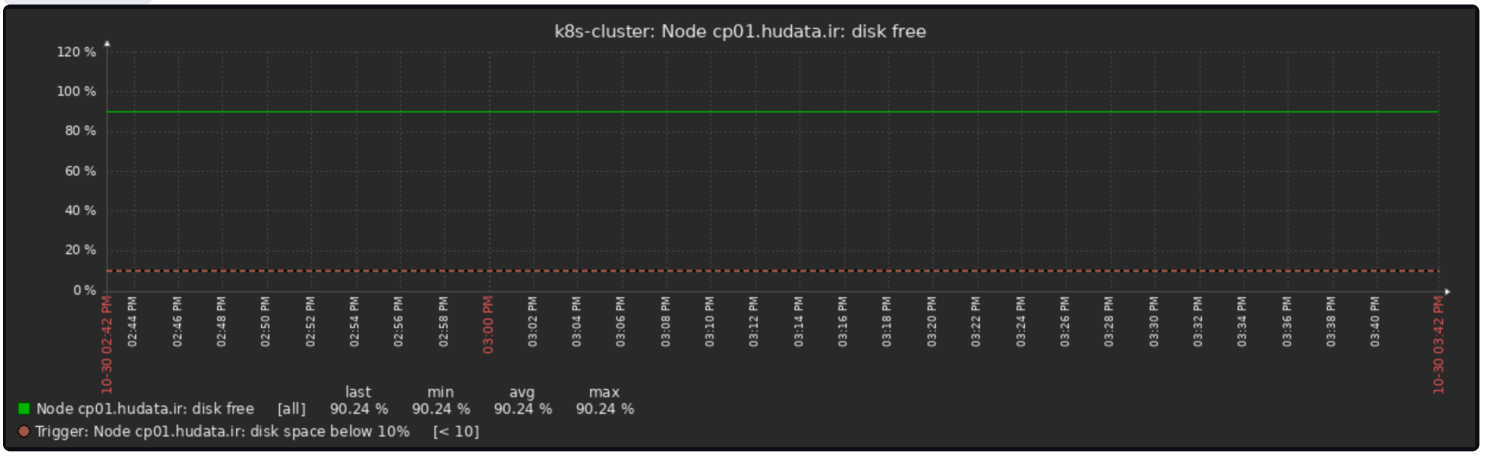
memory usage



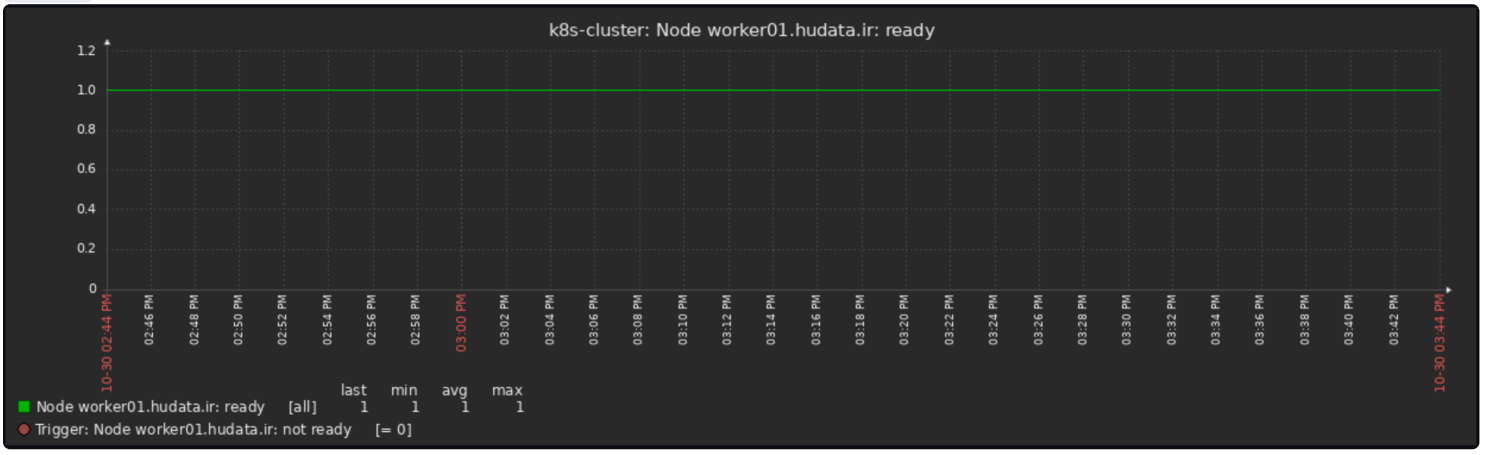
CPU usage



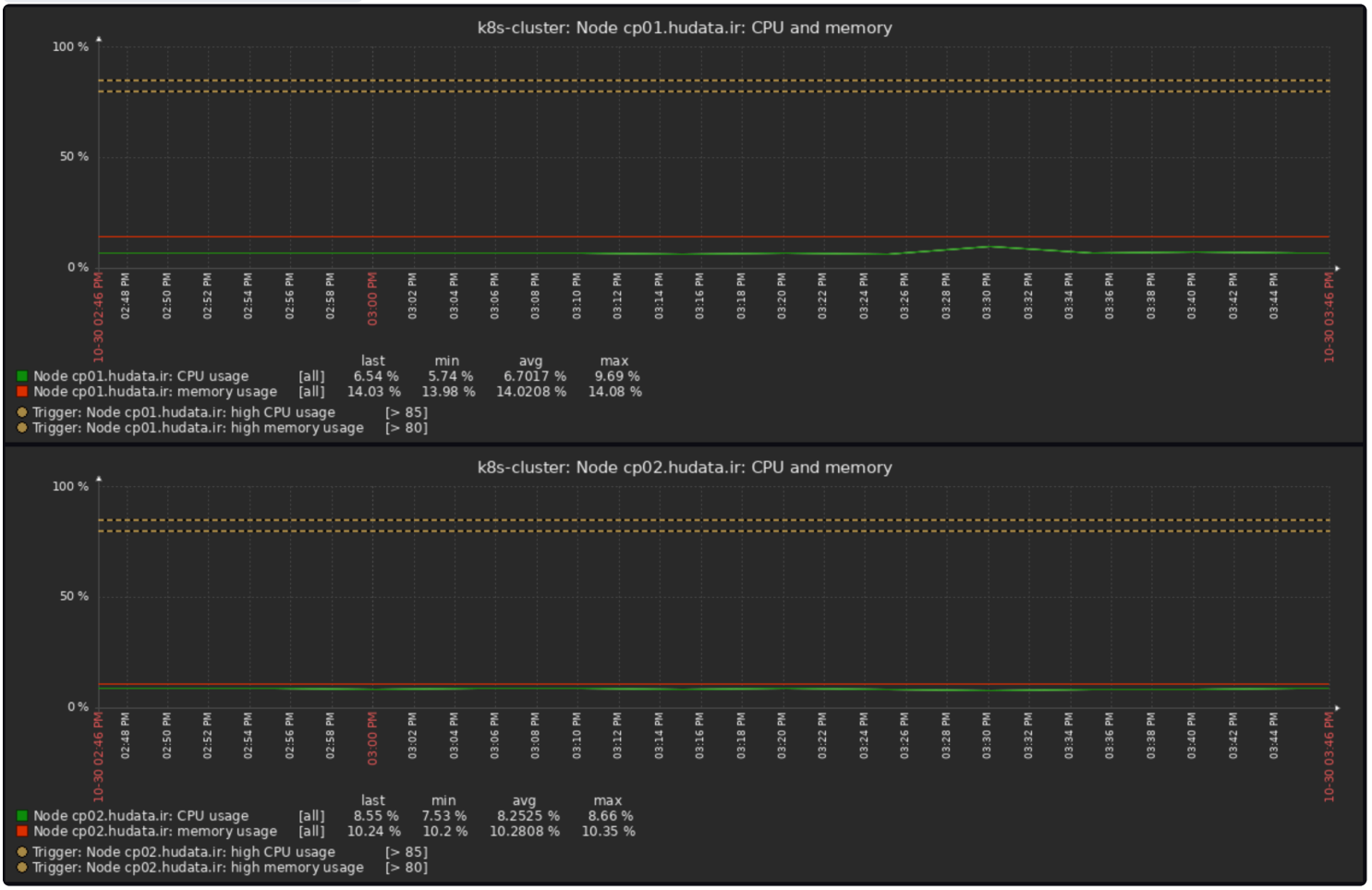
Disk free



ready



Comparison CPU and Memory



Pods by state

